

ADVANCED MULTI AI AGENT 2026

Agent Learning Hub

쓸모 있고 신뢰할 수 있는 AI 에이전트 학습 로드맵

출처: github.com/datawhalechina/Agent-Learning-Hub

코딩 에이전트

하네스 엔지니어링

Skills · MCP · 평가

왜 이 로드맵인가

링크 모음이 아니라 실행 todo list

- Datawhale 커뮤니티가 큐레이션한 **실행형 학습 로드맵** — 공식 블로그·논문·오픈소스·실전 경험 정리
- 초심자: **Learning Todo List**를 위→아래로 하나씩 체크
- LLM 앱 경험자: **Stage 2~3**부터 — agent loop·tool call·평가·공학 보강
- 프로젝트 지향: **Project Ladder** — 단계마다 돌아가는 작품 하나
- 핵심 원칙: **Build first, then read deeper**

에이전트란 무엇인가

chatbot · workflow · agent · multi-agent 구분

유형	정의	예
Chatbot	단발 질의응답	일반 대화
Workflow	정해진 순서 자동화	고정 파이프라인
Agent	스스로 도구 선택·행동·관찰 반복	코딩 에이전트
Multi-agent	여러 에이전트 협업·조율	research → write → review

기본 루프와 절제

observe → think → act, 그리고 언제 쓰지 말아야 하나

에이전트 루프

- **observe** — 관찰
- **think** — 판단
- **act** — 행동
- observe 재관찰로 순환
- 최대 스텝·타임아웃 필수

언제 쓰지 말아야 하나

- 과업이 예측 가능·프로세스 안정
- 일반 스크립트로 충분
- 에이전트는 불확실성만 늘림
- "agent 말고 workflow?"를 먼저 자문

지금 무엇을 배워야 하나 — 5대 우선순위

옛 crew / role-play 프레임워크에 올인하지 말 것

순위	무엇을	왜
1	코딩 에이전트 (Claude Code / Codex)	실코드·shell·편집·테스트·권한 = 최고의 공학 샘플
2	하네스 엔지니어링	능력의 상당수가 harness(도구·권한·상태·피드백·CI·eval)
3	개인 에이전트 (OpenClaw / Hermes)	장기실행·로컬우선·기억·skills·메시지 입구
4	Skills / MCP / A2A / ACP	능력 재사용·도구·에이전트·호스트 연결
5	평가와 안전	eval·trace·권한 없으면 데모일 뿐

우선순위 1.2 — 코딩 에이전트 & 하네스

코딩 에이전트

- Claude Code · Codex가 대표
- 실 코드베이스·shell·파일편집·테스트
- 권한·hooks·subagents·MCP
- 살아있는 에이전트 공학 표본

하네스 엔지니어링

- 모델 위에 얹는 시스템
- 도구 프로토콜·권한·상태
- 피드백·리플레이·트레이스
- CI·eval로 능력 유지

우선순위 3·4·5

PRIORITY 3

개인 에이전트

OpenClaw · Hermes: 로컬 우선·장기 기억
·skills·메시지 입구 — 개인 OS

PRIORITY 4

Skills · 프로토콜

Skills = 능력 재사용, MCP = 도구, A2A = 에이
전트, ACP = 호스트 연결

PRIORITY 5

평가 · 안전

고정 eval·trace·권한 경계 — 없으면 demo

최소 에이전트 루프 만들기

- LLM 대화 + **구조화 JSON** 출력
- 도구 함수 정의(search · calculator · read_file) + tool call 파싱
- 도구 실행 → 결과를 모델에 되먹임
- 최대 스텝 · 타임아웃 · 에러 처리
- **산출물: 50~150줄 최소 에이전트**

Tool Use · RAG · Memory

핵심 역량

- RAG: chunk · embed · retrieve · 인용
- 검색·DB·파일·브라우저·코드실행 도구화
- 단기 · 세션 · 장기 기억 구분
- 실패·빈결과·환각 인용 처리

참고 프로젝트

- GPT Researcher — 리서치 어시스턴트
- STORM · Onyx · RAGFlow — RAG
- mem0 · Letta — 기억
- 산출물: 자료 리서치 어시스턴트

현대 하네스 하나를 깊게

- 프레임워크 API가 아니라 **조직 방식** 학습(도구·컨텍스트·권한·상태·로그·서브태스크)
- 대상: Claude Code · learn-claude-code · claw0 · OpenClaw
- agent loop · tool registry · permission gate · session store · context compaction 찾기
- 최소 예제 실행 + 내 도구 하나 추가, trace 한 번 완전 해석
- **산출물: 디버깅 가능한 harness demo**

멀티 에이전트는 마법이 아니라 조율

- 역할 분담: **planner · executor · reviewer · critic · router**
- **supervisor · graph**로 관리 — 자유 잡담 금지
- 각 에이전트의 직무경계 · 입출력 schema · 정지조건 정의
- 루프 · 논쟁 · 과업 표류 · 컨텍스트 팽창 대응
- 산출물: **research → write → review → revise**

Skills · 프로토콜 · 능력 패키징

개념 구분

- **Tool** = 호출 인터페이스
- **Prompt** = 일회성 지시
- **MCP** = 외부 도구 · 데이터 연결
- **Skill** = 재사용 가능한 절차 지식

SKILL.md 구성

- name · description
- 언제 사용하나
- 단계 (steps)
- 검증 기준
- + 스크립트 · 템플릿 · 스모크 테스트

브라우저 · 컴퓨터 사용 에이전트

- browser agent ≠ 일반 API tool
- **Playwright · browser-use**로 관찰 · 클릭
- 안전 제한: 민감계정 로그인 금지 · 규칙 준수
- 페이지 변화 · 팝업 · 요소 탐지 실패 처리
- 스크린샷 · DOM · 액션 로그 기록
- **산출물: 공개 웹 전용 browser agent**

평가 · 관측 · 안전

- 고정 테스트셋 (데모 말고)
- 성공률 · 실패원인 · 톨호출수 · 비용 · 지연 기록
- **trace**로 실패 지점(prompt · 도구 · 검색 · 모델 · 상태) 파악
- 위험 도구엔 사람 확인 (메일 · 삭제 · 결제 · 게시)
- prompt injection · data exfiltration · tool abuse + 회귀 테스트
- 산출물: 20+ 태스크 eval 표

실제 에이전트 배포

- 명확한 사용자 · 태스크 · 성공기준
- 로그 · trace · 재시도 · 타임아웃 · 비용 상한
- **권한 경계** + 사람 확인
- 배포: CLI · Web · Slack bot · GitHub Action · 백그라운드
- README (실행 · 키 · 확장 · 한계)
- **산출물: 남이 clone해 돌리는 프로젝트**

프로젝트 사다리 — 11단계

기초 (1-6)

- 1 **Calculator** · 최소 tool loop
- 2 **Web Research** · 검색 · 인용
- 3 **PDF QA** · RAG
- 4 **Coding Review** · diff · 테스트
- 5 **Browser** · 관찰 · 복구
- 6 **Nano Claude Code** · shell · 권한 · compact

심화 (7-11)

- 7 **OpenClaw형 Gateway** · channel · memory
- 8 **Skill Pack** · SKILL.md · 테스트
- 9 **Multi-Agent Writer** · 협업
- 10 **Personal Agent** · 기억 · 메시지 입구
- 11 **Production Harness** · evals · CI · 리플레이

핵심 개념 한 장 정리

개념	무엇을 해결
Tool	호출 가능한 인터페이스
Skill	한 종류 과업의 절차 지식 · 스크립트 · 검증 패키지
Prompt	일회성 지시
MCP	외부 도구 · 데이터 표준 연결
A2A	에이전트 간 발견 · 통신 · 협업
ACP	에디터 · 터미널 · IDE · 호스트와 에이전트 통합

큐레이션 자료 지도

star 수 말고 학습 목적별로

레이어	대표	배움
Build From Scratch	learn-claude-code · claw0 · hello-agents	loop · registry · session · compaction
Personal	OpenClaw · Hermes · CyberClaw	장기실행 · skills · 기억 · 권한
Coding	Claude Code · Codex · OpenHands · SWE-agent	실코드 · shell · 테스트 · PR
RAG	GPT Researcher · LlamaIndex	검색 · rerank · 인용
논문	ReAct · Toolformer · Reflexion · SWE-bench	기반 범식 · 평가

Claude Code 심화 학습 경로

- ① 공식 문서 — hooks · subagents · MCP · GitHub Actions · permissions
- ② 복제 — **learn-claude-code**로 핵심 메커니즘 from scratch
- ③ 아키텍처 분석 — "Dive into Claude Code"로 harness 설계공간 추상화
- ④ 대조 — **hello-agents** · **OpenClaw** · **Hermes**로 공학 구현 비교

마무리 — 원칙과 오늘의 액션

8가지 학습 원칙

- 만들고 난 뒤 깊게 읽기
- 화려한 데모보다 작고 신뢰가능한 에이전트
- 엄격한 schema 도구
- 에이전트 늘리기 전에 eval
- 중요한 실행은 trace
- 멀티에이전트는 조율 문제
- 위험엔 사람 개입
- 플랫폼 규칙 · 저작권 존중

오늘의 액션

- 최소 에이전트 루프 한 개 만들기
- eval 표 20개 채우기
- 하네스 하나 깊게 파기
- 감사합니다 🙏

「멀티 AI Agent 개발과 활용」 2026

감사합니다

Thank You